Operating Systems CoSc3112_

June 15, 2025

Compiled by: Natan Asrat

https://www.linkedin.com/in/natan_asrat

Operating Systems

- Operating system: intermediary program between a computer user and the computer hardware
  - Time multiplexing: sharing CPU, printer…
  - Resource multiplexing: sharing main memory…
  - Resource allocator: manages and allocates resources.
  - Control program: controls the execution of user programs and operations of I/O devices.
  - Layers:
    - Inner layer, computer hardware
    - Middle layer, operating system
    - Outer layer, different software

Operating Systems

- Operating system - Main functions:
  - Multiprogramming, multiprocessor, Computer resource management
  - Provides a user interface, Runs software utilities and programs
  - Schedule jobs, Enforce security measures
  - Provide tools to configure the operating system and hardware
  - Administers user actions and accounts, Maintain system integrity, handle failures
  - Give users and processes an interface, Provide convenient persistent storage (files)

Operating Systems

- Operating system - Features:
  - Authentication of users: password, passphrase comparison, biometrics, digital authentication (SSL, CA, PKI, Kerberos, DS)
  - Mandatory Access Control (multilevel security): classifying data and users into various security classes
  - Discretionary Access Control: grant privileges to users
  - Protection of memory: user space, paging, segmentations

Operating Systems

- Operating system - Features:
  - File and I/O device access control: access control matrix
  - Enforcement of sharing resources: preserve integrity, consistency (critical section)
  - Fair service: no starvation and deadlock
  - Inter-process communication & synchronization: shared variable (e.g, using semaphores)
  - Protection of data: encryption, isolation

Operating Systems

- Operating system:
  - Computer System Components: Hardware, Operating system, Applications programs, Users (people, machines, other computers).
  - Mainframe OS: processing of many jobs at once, automatically transfers control from one job to another

Operating Systems

- Operating system - Mainframe OS - Services:
  - Batch system: processes routine jobs in the absence of interactive user.
  - Multi programmed Batch Systems: jobs are kept in main memory at the same time, and the CPU is multiplexed among them; requires I/O routines, Memory management, CPU scheduling, Allocation of resources
  - Transaction processing system: handles large number of small requests
  - Time sharing system (interactive computing):
    - CPU is multiplexed among several jobs that are kept in memory and on disk
    - On-line communication between the user and the system

Operating Systems

- Operating system:
  - Desktop Systems (Personal Computers): computer system dedicated to a single user
    - individuals have sole use of computer and do not need advanced CPU utilization
    - may run several different types of operating systems
  - Parallel Systems (Multiprocessor OS): connecting multiples CPU’s into a single system.
    - Tightly coupled system: processors share bus, memory, clock and peripheral device.
    - communication usually takes place through the shared memory.
    - Advantages: Increased throughput, Economical (share mass storage), Increased reliability (If one fail, the other will take responsibility)

Operating Systems

- Operating system:
  - Distributed Systems: distribute the computation among several physical processors.
    - Loosely coupled system: each processor has a local memory, clock, peripheral devices,…
    - communication through communications lines, (high-speed buses or telephone lines)
    - Advantages: Resources Sharing, Sharing load, Reliability, Communications
    - Requires networking infrastructure; LAN or WAN; Client-server or peer-to-peer

Operating Systems

- Operating system:
  - Real-Time Operating Systems: often a control device in a dedicated application; time is a key parameter
  - Embedded Operating Systems: Personal Digital Assistants (PDAs), Cellular telephones, Windows CE (Consumer Electronics), PalmOS, Android OS, …
    - Issues: Limited memory, Slow processors, Small display screens
  - Contemporary OS: prioritizes convenience, efficiency, and the ability to evolve

Operating Systems

- Operating system - Contemporary OS
  - Convenience:
    - User Interface (user friendly GUI),
    - Application Support (platform runs many apps),
    - File Management (directories, folders, and file navigation interfaces),
    - Device integration (plug and play, connect and use devices like printers)

Operating Systems

- Operating system - Contemporary OS
  - Efficiency:
    - Resource Management (optimal performance, manage memory, processor, and storage..),
    - Memory Management (handles memory allocation and deallocation),
    - Multitasking (concurrent execution of multiple apps or processes),
    - I/O Management (allowing devices to communicate with the computer system)

Operating Systems

- Operating system - Contemporary OS
  - Ability to Evolve:
    - Compatibility (allowing users to upgrade their systems without losing compatibility with existing applications),
    - Updates and Patches (vendors frequently release updates and patches to address vulnerabilities),
    - Modularity (allowing components to be updated or replaced independently),
    - Support for New Technologies (incorporate support for emerging technologies i.e. cloud, virtualization, AI, IoT)

Operating Systems

- Process Management: fundamental task of modern operating systems
  - Includes:
    - Creation of processes, Allocation of resources to processes, Managing conflicting demands
    - Protecting resources of each processes from other processes
    - Enabling processes to share and exchange information
    - Enabling synchronization among processes for sequencing & coordination (dependencies)
    - Termination of processes

Operating Systems

- Process Management - Process vs. Program
  - Program: sequence of instructions defined to perform some task; a passive entity
  - Process: a program in execution; instance of a running program; an active entity
  - A processor performs the actions defined by a process

Operating Systems

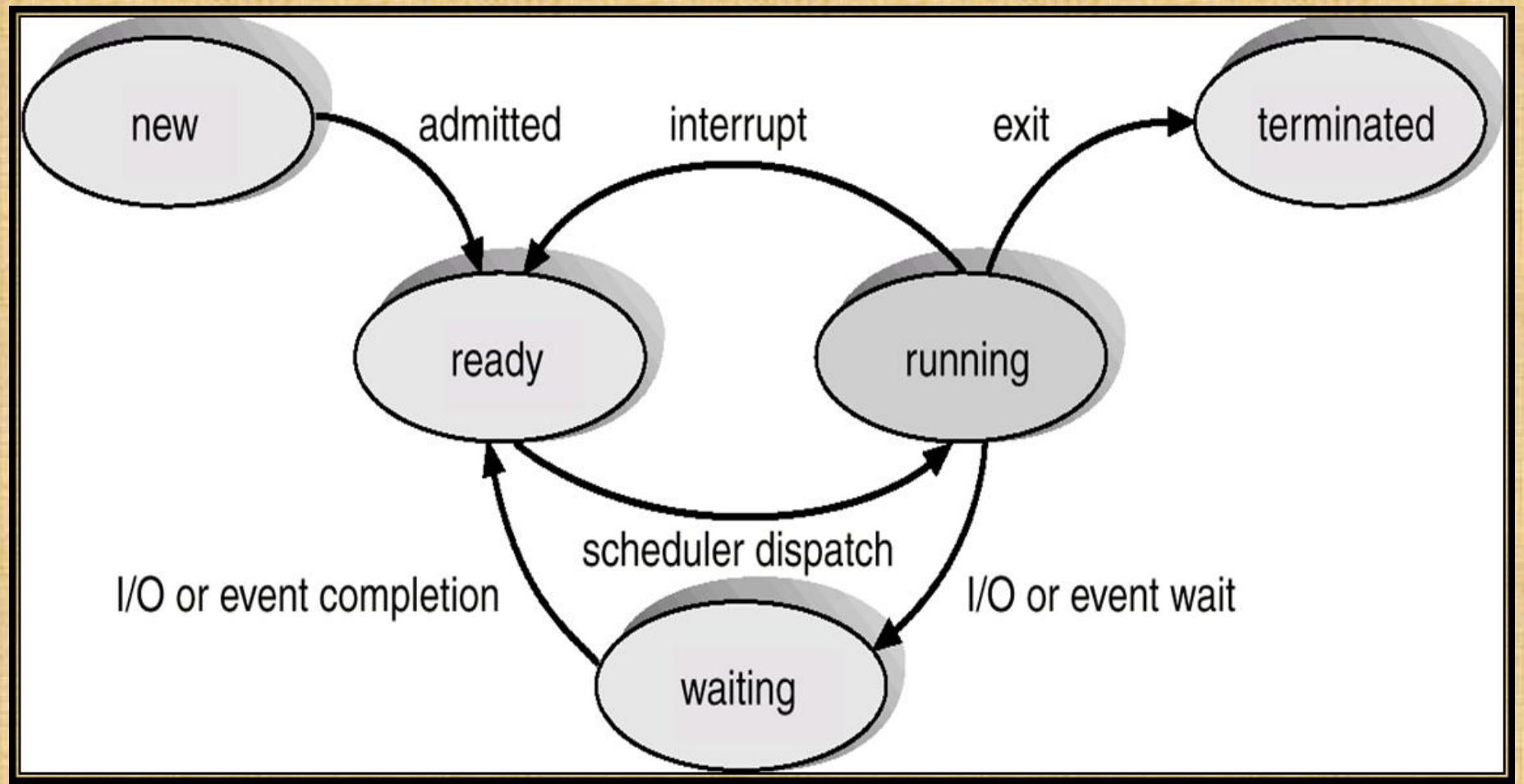
- Process Management - Types of Processes:
  - Sequential Processes: execute one after the other; at most one process is being executed; traditional single-threaded programs or sequential algorithms
  - **Concurrent Processes**
    - ***True Concurrency*** (Multiprocessing): two or more processes are executed simultaneously in a multiprocessor environment (real parallelism)
    - ***Apparent Concurrency*** (Multiprogramming): pseudo-concurrency or simulated concurrency, executed in parallel in a uniprocessor environment by switching from one process to another (pseudo parallelism)

Operating Systems

- Process Management - Process States:
  - New: process just been created but has not yet been admitted to the pool of executable processes by the operating system.
    - information about process is already maintained in memory but the code is not loaded and no space has been allocated for the process.
  - Ready: process not currently executing but is ready as soon as the OS dispatches it.
    - process is in main memory and is available for execution.
  - Running: process is currently being executed.
  - Blocked (Waiting): process is waiting for the completion of some event, such as I/O

Operating Systems

- Process Management - Process States:



Operating Systems

- Process Management:
  - Process Control Block (task control block) - PCB: how a process is represented in the OS

Operating Systems

- Process Management – PCB Information
  - **Process state**: new, ready, running and waiting, halted
  - **Program counter**: address of the next instruction to be executed for this process.
  - **CPU registers**: depends on the computer architecture
    - accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information.
  - **CPU-Schedule information**: priority, pointers to scheduling queues, scheduling params
  - **Memory-management information**: value of the base and limit registers, the page tables or the segment tables, (depends on the memory system of the OS)

Operating Systems

- Process Management:
  - Threads: dispatchable unit of work (lightweight process)

Operating Systems

- Process Management - Threads
  - has independent context, state and stack
  - process is a collection of one or more threads and associated system resources
  - traditional operating systems are single-threaded systems.
  - Types of threads
    - User threads: supported above the kernel; implemented by a thread library at the user level.
    - Kernel threads: supported directly by the OS; performs thread creation, scheduling, and management in kernel space

Operating Systems

- Process Management - Threads
  - **Multithreading:** a process, executing an application, is divided into threads that can run concurrently.
    - Modern operating systems are multithreaded systems.

Operating Systems

- Process Management - Threads - Multithreading benefits
  - **Responsiveness**: allow a program to continue running even if part of it is blocked or is performing a lengthy operation; increases responsiveness to the user.
  - **Resource sharing**: by default, threads share the memory and the resources of the process to which they belong.

Operating Systems

- Process Management - Threads - Multithreading benefits
  - **Economy**: **a**llocating memory and resources for process creation is costly; threads share resources of the process to which they belong; it is more economical to create and context switch threads
  - **Utilization of multiprocessor architectures**: each thread may be running in parallel on a different processor; a single-threaded process can only run on one CPU

Operating Systems

- CPU Scheduling
  - CPU Scheduler: selects among ready processes and allocates CPU to one of them
  - CPU scheduling decisions when:
    - Switching from running to waiting state (non-preemptive)
    - Switching from running to ready state (preemptive)
    - Switching from waiting to ready (preemptive)
    - Terminating (non-preemptive)

Operating Systems

- CPU Scheduling - CPU Scheduling Criteria
  - Maximize throughput: ****run as many jobs as possible in a given amount of time.
    - by running only short jobs or by running jobs without interruptions.
  - Minimize response time: quickly turn around interactive requests.
    - by running only interactive jobs and letting batch jobs wait until the interactive load ceases.
  - Minimize turnaround time: move entire jobs in and out of the system quickly.
    - by running all batch jobs first (because batch jobs can be grouped to run more efficiently than interactive jobs).

Operating Systems

- CPU Scheduling - CPU Scheduling Criteria
  - Minimize waiting time: move jobs out of the READY queue as quickly as possible.
    - by reducing the number of users allowed on the system.
  - Maximize CPU efficiency: ****keep the CPU busy 100 percent of the time.
    - by running only CPU-bound jobs (and not I/O-bound jobs).
  - Ensure fairness for all jobs: give everyone an equal amount of CPU and I/O time.
    - by not giving special treatment to any job, regardless of its processing characteristics or priority.

Operating Systems

- CPU Scheduling - Process Scheduling Algorithms
  - Process Scheduler relies on a process scheduling algorithm; based on a specific policy, to allocate the CPU and move jobs through the system

Operating Systems

- CPU Scheduling - Process Scheduling Algorithms
  - Algorithms:
    - First-Come, First-Served (FCFS): non preemptive, uses FIFO queue, no wait queues in strict versions (each job runs to completion)
    - Shortest-Job-First (SJR): associate length of CPU bursts for each process,
      - non-preemptive: can't be preempted until process completes its CPU burst
      - preemptive: preempt if a new process arrives with less CPU burst length than the current running process's remaining time
      - SJF is optimal: gives minimum average waiting time for a given set of processes

Operating Systems

- CPU Scheduling - Process Scheduling Algorithms
  - Algorithms:
    - Priority Scheduling: priority number (integer) is associated with each process
      - SJF is a priority scheduling where priority is the predicted next CPU burst time
      - Problem: Starvation – low priority processes may never execute
        - Solution: Aging – as time progresses increase the priority of the process

Operating Systems

- CPU Scheduling - Process Scheduling Algorithms
  - Algorithms:
    - Round Robin (RR): each process gets a small unit of CPU time (*time quantum*)
      - After time has elapsed, process is preempted and added to the end of the ready queue.
      - If there are *n* processes in the ready queue and the time quantum is *q*, then each process gets *1/n* of the CPU time in chunks of at most *q* time units at once.
      - No process waits more than (*n -1*) * *q* time units.
      - Performance: if *q is* large it becomes a FIFO; if *q is* small context switching overhead is too high.

Operating Systems

- Inter Process Communication
  - Process need to communicate with other processes.
  - Race Condition: situation where several processes access – and manipulate shared data concurrently (main memory, printer…)
    - final value of the shared data depends upon which process finishes last
    - to prevent race conditions, concurrent processes must be synchronized

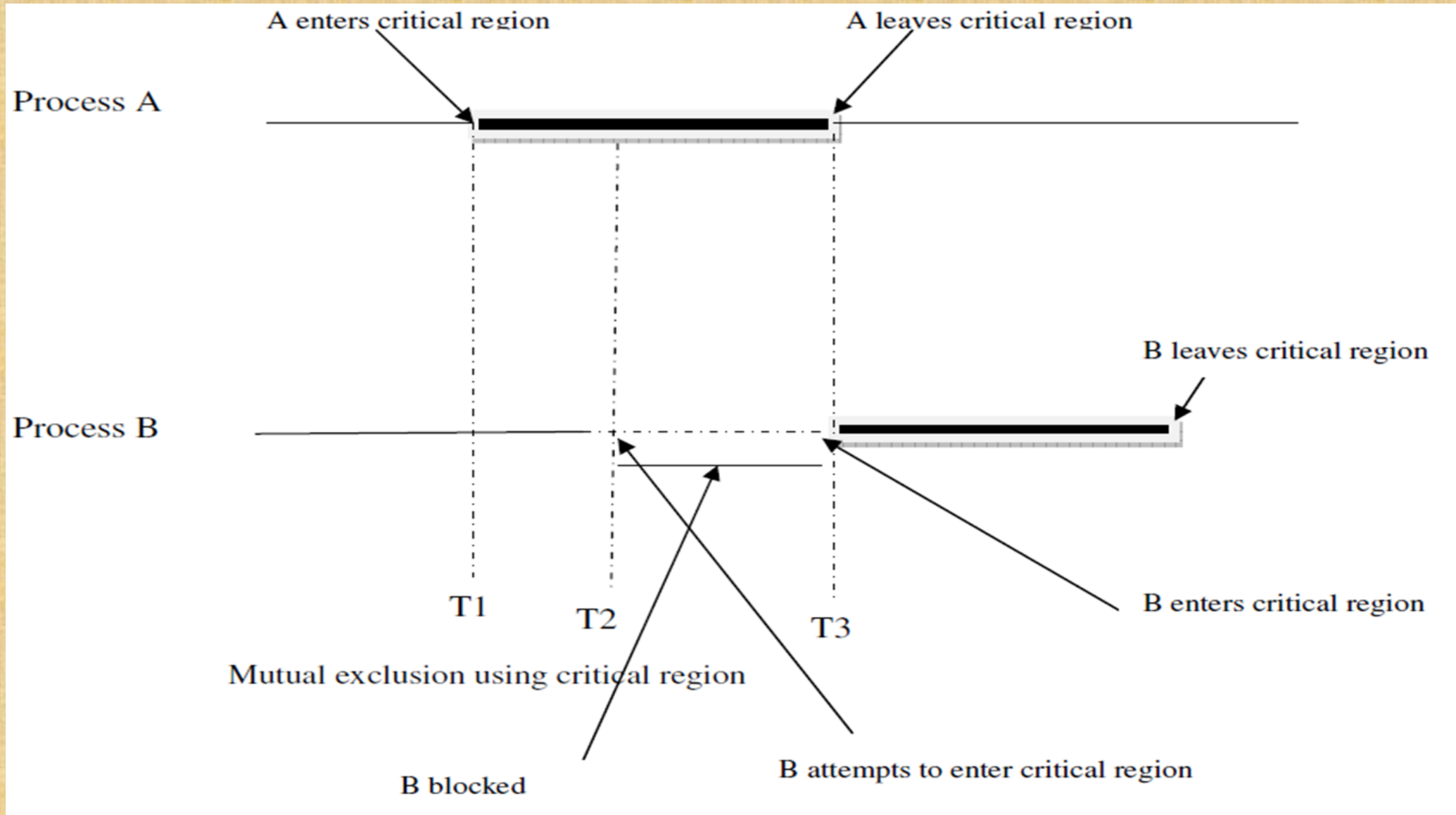
Operating Systems

- Inter Process Communication
  - The Critical-Section Problem: How do we avoid race conditions?
    - Critical region: portion of code where a process accesses **shared resources** like memory, files, or devices. If multiple processes enter their critical sections at the same time, it can lead to **race conditions**

Operating Systems

- Inter Process Communication - Critical-Section
  - Solution: Mutual Exclusion
    - No two processes may be simultaneously inside the critical regions.
    - No assumptions may be made about speeds or the number of CPUs.
    - No process running outside its critical region may block other processes.
    - No process should have to wait forever to inter its critical region.

Operating Systems



Operating Systems

- Inter Process Communication - Critical-Section
  - Algorithms: Disabling Interrupts, Lock Variables, Strict Alternation, Semaphores, Monitors, Message passing

Operating Systems

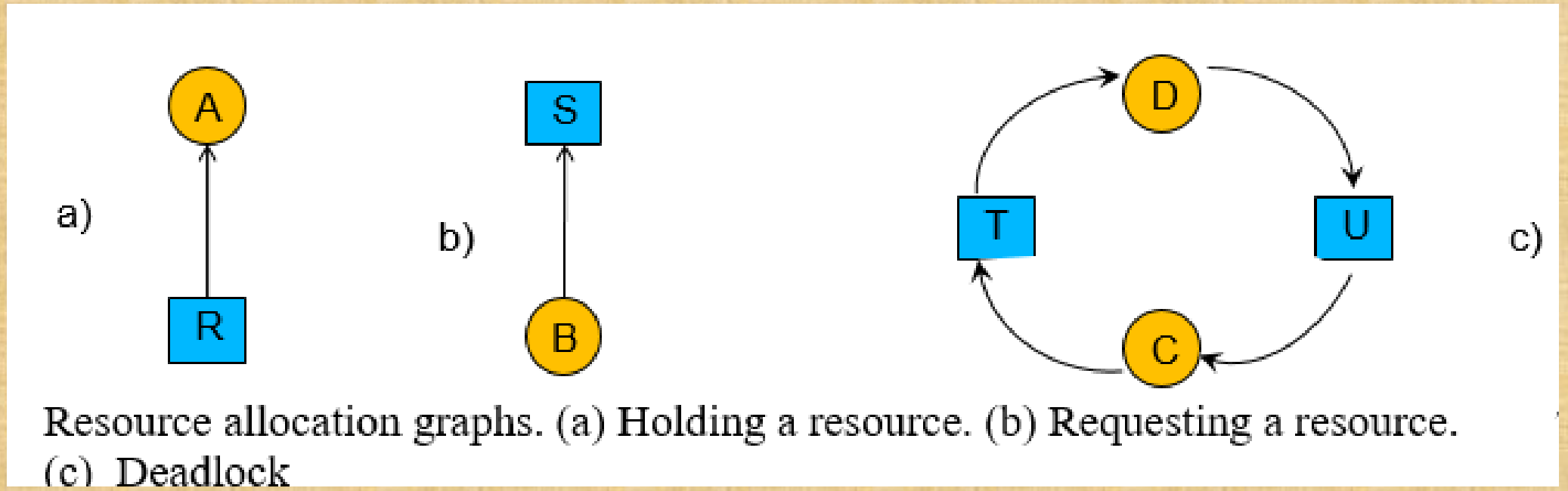
- Deadlock: a process needs exclusive access to several resources; it holds one resource and waits till another resource is released, and this other resource is held by a process waiting for the first resource to be released
  - None of the processes can run, none of them can release any resources, and all the processes continue to wait forever

Operating Systems

- Deadlock - Conditions of a deadlock (all required):
  - Mutual exclusion condition: resource is assigned to one process at most.
  - Hold and wait condition: processes can request new resources without releasing other resources granted earlier
  - No preemption condition: resources once granted cannot be forcibly taken away, must be explicitly released by the process holding them.
  - Circular wait condition: circular chain of two or more processes, each of which is waiting for a resource held by the next member of the chain

Operating Systems

- Deadlock:
  - Deadlock modeling: modeled using directed graphs



Resource allocation graphs. (a) Holding a resource. (b) Requesting a resource. (c) Deadlock

Operating Systems

- Deadlock:
  - If granting a particular request might lead to deadlock, the OS can suspend the process with out granting the request (don’t schedule the process until it is safe)

Operating Systems

- Deadlock - Strategies to deal with deadlocks:
  - Just ignore the problem altogether **- Ostrich Algorithm
  - Detection and recovery: Let deadlocks occur, detect them, and take action.
    - Detection: with One Resource of Each Type, with Multiple Resources of Each Type
    - Recovery: through Preemption, through Rollback, through Killing Processes
  - Dynamic avoidance: careful resource allocation; whether granting a resource is safe or not
    - Algorithms: Safe and Unsafe States, The Bankers algorithm
  - Prevention: by structurally negating one of the four conditions of a deadlock.